Technical Design Document & Product Requirements: SwiftList & CSG Unified Platform

Version: 1.7 (AWS 3-Tier Enterprise Deployment)
Status: Approved for Core Infrastructure Build
Governance: CSG Unified C-Suite Board

---

1. Executive Summary: The Merchant Intelligence Network

1.1 Strategic Vision and Product Philosophy

The digital commerce landscape is currently saturated with "GenAI Wrappers"—thin user interfaces that commoditize access to large language models (LLMs) without adding structural defensibility. **SwiftList** represents a fundamental strategic pivot from this fragility toward a robust **Merchant Intelligence Network**.

The objective is not merely to facilitate listing creation but to engineer a proprietary "Data Moat" where every user interaction refines the system's "Ground Truth" (The Style Library). This transformation is governed by the rigorous financial discipline of the **CSG Unified Platform**, which mandates a transition from operational opacity to a margin-guarded ecosystem targeting **60% Net Margins**.

1.2 The "Professional Maker" and the Guilt Factor

Product requirements are derived from a deep psycho-demographic analysis of the target persona: the "Pro Seller." Unlike the "Hobbyist" or the volume "Agency," the Pro Seller is a solitary entrepreneur managing complex inventory. The core psychological driver identified is "The Guilt Factor"—the pervasive anxiety that time spent on administrative tasks (SEO, data entry, photography) is time stolen from the creative act of sourcing or making.

SwiftList's value proposition is the **Recovery of Creative Agency**. The platform functions as an "Asset Generator" that creates a "Single Player" utility so profound that it works immediately, while simultaneously building the "Style Library" network effect that bonds users to the platform via shared, royalty-generating aesthetic presets.

1.3 Architectural Pivot: AWS 3-Tier & Hybrid Storage

Version 1.5 officially transitions all customer-facing compute to AWS to ensure 99.99% uptime

and low-latency asset delivery.

- **Tier 1 (Presentation): AWS Amplify**. Global CDN for lightning-fast site loads and marketplace-standard UI.
- **Tier 2 (Logic): AWS Lightsail** (4GB/2-vCPU). High-speed, dedicated n8n orchestration.
- **Tier 3 (Truth): Amazon RDS (Postgres)**. Mission-critical user, credit, and project persistence.
- **Hybrid Storage: Amazon S3 Intelligent-Tiering** handles the "Studio" project library, while Local NUCs serve as a "Cold Vault" backup tier via **AWS DataSync**.

1.4 Financial Governance & The Margin Rule

Every workflow is bound by the "60% Net Margin" goal.

- **Tier 1 (Utility):** Executed on AWS Lightsail + Gemini 2.0 Flash.
- **Tier 2 (Creativity):** Utilizes **OpenAI GPT-Image-1.5** via OpenRouter (~$0.04/gen).
- **Royalties:** A closed-loop "Surcharge Model" ensures creator payouts do not eat into platform margins.

---

2. System Architecture: The V1.5 Hybrid Model

2.1 Architectural Overview

The system now operates as a distributed Hybrid Cloud environment:

- **The Vibe (Frontend):** Hosted on **AWS Amplify**. Implements Skeleton Loaders (pulsating grey gradient #f0f0f0 to #e0e0e0) for long-running generative tasks.
- **The Factory (Compute):** Hosted on **AWS Lightsail**. Runs n8n in Docker. This server is the primary executor for all 15 core workflows.
- **The Studio (Storage): Amazon S3**. Assets automatically transition from High-Speed Frequent Access to Deep Archive (saving 68–99% cost) after user inactivity.
- **The Vault (Backup): Local NUC Cluster**. Daily sync of all user assets and database snapshots to on-prem storage for extreme data durability.

2.2 System Visualization (Mermaid.js)

Code snippet

```
graph TD
  subgraph "Tier 1: Presentation (Amplify + S3)"
    Amplify_Site
```

```
    S3_Studio
  end
  subgraph "Tier 2: Business Logic (Lightsail)"
    n8n_Engine[n8n Orchestrator]
    Docker_Nodes
  end
  subgraph "Tier 3: Data Access (RDS)"
    Postgres_DB
  end
  subgraph "External Factory"
    Gemini
    GPT_Img
    Runway
  end
  subgraph "The Cold Vault (On-Prem)"
    NUC_Cluster[3x Intel NUCs]
    DataSync
  end

  Amplify_Site -->|HTTPS/Secure| n8n_Engine
  n8n_Engine -->|Query/Write| Postgres_DB
  n8n_Engine -->|Fetch/Store| S3_Studio
  n8n_Engine -->|Execute| External_Factory
  S3_Studio -.->|Sync| DataSync
  DataSync -.->|Archival| NUC_Cluster
```

---

3. Technology Stack & Decision Matrix (v1.5)

3.1 The "Golden Stack" Selection

Component	Technology	Role & Rationale
Frontend	**AWS Amplify**	Global CDN distribution for the Stitch-generated React/Vue app.
Orchestrator	**AWS Lightsail**	Dedicated 4GB/2-vCPU instance running Dockerized n8n for stable

		orchestration.
Database	**Amazon RDS**	Managed Postgres instance for high availability and automated backups.
AI (Tier 1)	**Gemini 2.0 Flash**	Native vision/multimodal handling for near-zero cost image analysis.
AI (Tier 2)	**GPT-Image-1.5**	Precise conversational in-painting for product-perfect edits ($0.04/gen).
AI (Tier 3)	**Gemini 2.5 Pro**	Native video understanding (removes Whisper API dependency) and high-context (1M token) scripting.
Video Engine	**Runway Gen-3**	Alpha Turbo model preferred for cost/speed efficiency in premium video generation.
Backup	**AWS DataSync**	Automated bridge to the on-premise NUC "Cold Vault."

---

4. Database Schema: The "Truth Layer"

4.1 Schema Philosophy

The database enforces the "Financial Truth" and "Integrity Truth." Version 1.5 adapts the schema to reflect the AWS infrastructure and updated model defaults.

4.2 Comprehensive SQL Definitions

4.2.1 Profiles Table (User Identity & Kill-Switch)

SQL

```sql
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT UNIQUE NOT NULL,
  swift_credits INTEGER DEFAULT 100 CHECK (swift_credits >= 0),
  earned_credits INTEGER DEFAULT 0,
  tier_level TEXT DEFAULT 'STARTER',
  growth_pool_contribution INTEGER DEFAULT 0,
  is_banned BOOLEAN DEFAULT FALSE, -- Integrity Kill-Switch
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

  -- Constraint: Prevent negative balances during refunds
  CONSTRAINT earned_credits_non_negative CHECK (earned_credits >= 0)
);
```

4.2.2 Jobs Table (Latency & COGS Tracking)

SQL

```sql
CREATE TABLE public.jobs (
  job_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id UUID REFERENCES public.orders(order_id),
  worker_id TEXT, -- References Lightsail Docker Container ID
  job_status TEXT DEFAULT 'QUEUED',
  result_payload JSONB,
  error_log TEXT,

  -- v1.5 Tracking
  model_engine TEXT DEFAULT 'gemini-2.0-flash',
  estimated_cogs_usd DECIMAL(10, 5),
  processing_latency_ms INTEGER,

  started_at TIMESTAMP WITH TIME ZONE,
  completed_at TIMESTAMP WITH TIME ZONE
);
```

4.2.3 Presets & Royalties

SQL

```sql
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE public.presets (
  preset_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id UUID REFERENCES public.profiles(id),
  asset_name TEXT NOT NULL,
  style_vector vector(1536),
  is_public BOOLEAN DEFAULT FALSE,
  total_usage_count INTEGER DEFAULT 0,
  unique_users_count INTEGER DEFAULT 0,
  usat_score DECIMAL(3, 2) DEFAULT 1.00,
  integrity_flag BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

4.2.4 Transactions (Financial Ledger)

SQL

```sql
CREATE TABLE public.transactions (
  transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.profiles(id),
  amount_change INTEGER NOT NULL,
  transaction_type TEXT CHECK (transaction_type IN ('PURCHASE', 'JOB_SPEND',
'ROYALTY_SURCHARGE', 'REFUND', 'GROWTH_POOL_DIVERT')),
  related_preset_id UUID REFERENCES public.presets(preset_id),
  ip_address INET,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Performance Optimization for Integrity Checks
CREATE INDEX idx_transactions_ip_created ON public.transactions (ip_address, created_at);
```

4.2.5 System Audits (Panopticon Mode)

SQL

```sql
CREATE TABLE public.system_audits (
 id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
 audit_date DATE DEFAULT current_date,
 efficiency_score DECIMAL(3, 2),
 pattern_analysis TEXT,
 recommended_revisions JSONB,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

4. Database Schema: The "Truth Layer" (v1.6 Updates)

The SQL definitions in this section have been updated to support the new Creator Economy and Job Refinement flows.

4.2.1 Profiles Table (User Identity & Kill-Switch)

None

```
CREATE TABLE public.profiles (

  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE
CASCADE,

  email TEXT UNIQUE NOT NULL,

  swift_credits INTEGER DEFAULT 100 CHECK (swift_credits >= 0),

  earned_credits INTEGER DEFAULT 0,

  lifetime_credits_earned INTEGER DEFAULT 0, -- v1.6: Creator
Economy

  current_month_earnings INTEGER DEFAULT 0,  -- v1.6: Creator
```

```
Economy

  tier_level TEXT DEFAULT 'STARTER',

  growth_pool_contribution INTEGER DEFAULT 0,

  is_banned BOOLEAN DEFAULT FALSE, -- Integrity Kill-Switch

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),


  -- Constraint: Prevent negative balances during refunds

  CONSTRAINT earned_credits_non_negative CHECK (earned_credits >=
0)

);
```

4.2.2 Jobs Table (Latency & COGS Tracking)

```
None
CREATE TABLE public.jobs (

  job_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  order_id UUID REFERENCES public.orders(order_id),

  parent_job_id UUID REFERENCES public.jobs(job_id), -- v1.6: Job
Refinement Iteration

  worker_id TEXT, -- References Lightsail Docker Container ID

  job_status TEXT DEFAULT 'QUEUED',

  result_payload JSONB,

  error_log TEXT,
```

```
  -- v1.5 Tracking
  model_engine TEXT DEFAULT 'gemini-2.0-flash',
  estimated_cogs_usd DECIMAL(10, 5),
  processing_latency_ms INTEGER,

  started_at TIMESTAMP WITH TIME ZONE,
  completed_at TIMESTAMP WITH TIME ZONE
);
```

4.2.3 Presets & Royalties

```
None
CREATE EXTENSION IF NOT EXISTS vector;


CREATE TABLE public.presets (
  preset_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id UUID REFERENCES public.profiles(id), -- v1.6:
Explicit Foreign Key
  asset_name TEXT NOT NULL,
  style_vector vector(1536),
  is_public BOOLEAN DEFAULT FALSE,
  visibility TEXT DEFAULT 'private' CHECK (visibility IN
('private', 'public')), -- v1.6: Marketplace visibility
  usage_count INTEGER DEFAULT 0, -- v1.6: Total times the preset
was used
```

```
  unique_users_count INTEGER DEFAULT 0,
  usat_score DECIMAL(3, 2) DEFAULT 1.00,
  integrity_flag BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

---

5. API Integration Dictionary

5.1 Tier 2: Conversational In-Painting

- **Model:** openai/gpt-image-1.5 (via OpenRouter).
- **Cost:** ~$0.04 per generation.
- **Use Case:** Precise editing (e.g., "Add a logo," "Change lighting").

5.2 Lifeguard Audit Endpoint

- **Endpoint:** Internal n8n Cron (Running on AWS Lightsail).
- **Trigger:** Daily at 00:00 UTC.
- **Action:** Gemini 2.5 Pro analyzes public.jobs logs. It ingests error logs and latency data to identify patterns and writes a report to public.system_audits.

---

6. The Lifeguard Protocol: Integrity & Resilience

6.1 Anti-Gaming KPIs (The "Wash Trading" Monitor)

1. **Unique Usage Ratio (UUR):** Threshold < 0.05 flags "Wash Trading."
2. **IP Collision Protocol:** >5 distinct User IDs from 1 IP in 1 hour triggers a freeze.
3. **Credit Circularity:** Reciprocal usage loops pause payouts.

6.2 Self-Healing (Auto-Refunds)

- **Trigger:** Job Status = FAILED or TIMEOUT.
- **Action:** Lifeguard executes refund_credits + 1 Bonus Credit.

---

7. UX/UI Specifications: The "Vibe" Doctrine

7.1 Visual Behaviors

- **Skeleton Loaders:** Mandatory pulsating grey gradient (#f0f0f0 to #e0e0e0) for all generative wait states.
- **Debounced Search:** 300ms delay, min 3 chars.

7.2 Easter Eggs: The "Vibe Code"

- **Requirement:** Embed `` in index.html.
- **Reward:** 50 Credits for discovery.

7.3 Growth & Network Effects (v1.5)

- **Referral Engine:** Dual-sided bonus credit logic.
  - **Trigger:** New Account conversion.
  - **Action:** +5 Credits to both the Referrer and the Referee.
- **Search Doctrine:** Global User Search with debounced autocomplete to find Creators and Presets.
- **Profile Permissions:**
  - Public profiles feature a **"Follow"** action.
  - Edit/Delete actions are strictly restricted to the owner (Row Level Security).
- **The "eBay Standard":** All workflows default to **1:1 aspect ratio**, **1500px resolution**, **white background**, and marketplace-compliant metadata injection.

---

8. Royalty Financial Logic: Surcharge Model

8.1 The "Style Surcharge"

- **Base:** 10 Credits.
- **Surcharge:** +1 Credit (Public Presets).
- **Total:** 11 Credits.
- **Payout:** +1 Credit to Creator.

8.2 Caps & Overflow

- **Hard Cap:** 5,000 Credits/mo.
- **Overflow:** Diverted to "Growth Pool."
- **Constraint:** earned_credits cannot be negative.

---

9. Analytics & KPI Framework

- **Integrity:** UUR > 0.20.
- **Performance:** Latency < 60s (Tier 2).
- **Financial:** Net Margin 60% (Goal).

---

10. Implementation Roadmap

Phase 1: The "Smart Assistant" (MVP)

- **Tech:** AWS Amplify + Lightsail + RDS.
- **Goal:** 50 Pro Sellers.

Phase 2: The "Style Library" (Growth)

- **Feature:** Preset Builder & pgvector.
- **Integrity:** Activate IP Collision Index & "Panopticon" Audits.

Phase 3: The "Verified Guild" (Authority)

- **Feature:** Tier 3 Video (Gemini 2.5 Pro / Runway).
- **Scale:** Scale Lightsail instances or migrate n8n to AWS Fargate if load exceeds capacity.

11. The "Panopticon" Admin Dashboard (Internal)

You need a "God View" of the system that bypasses the customer-facing UI.

🏛️ TDD/PRD v1.5: The "Mission Control" Dashboard Update

Please add this section to your **"Admin & Management"** chapter in the TDD/PRD.

Backend Dashboard Architecture (The Mission Control)

The SwiftList Admin infrastructure shall be decoupled into three logical monitoring views to ensure operational transparency and margin protection.

11.1 The Financial Pulse (Operational Truth)

- **Real-Time Net Margin Gauge:** Calculates $(Revenue - Variable COGS) / Revenue$. Threshold: A dip below **50%** triggers an automated alert to the C-Suite.
- **Credit Breakage & Churn:** Tracks the volume of unused credits to identify user abandonment early.
- **LTV:CAC Ratio:** Monitors if the "Pro" subscription revenue justifies the **$5.00**

Blended CAC.

11.2 The Engineering Health Monitor (TTA Logic)

- **Time-to-Asset (TTA):** Measures the latency from job submission to final proof. **Target: < 60 seconds**.
- **Hybrid Load Split:** Visualizes the ratio of tasks processed by **Local NUCs (Tier 4)** vs. **AWS Lightsail/Fargate (Tier 2)**.
- **The Integrity Guard:** A real-time feed of **IP Collisions** and **Unique Usage Ratios (UUR)** to stop account syphoning and wash-trading.

11.3 The Network Effect (NFX) Engine

- **Style Remix Rate:** Percentage of jobs utilizing a **Style Library Preset**.
- **First-Pass Yield (FPY):** Tracking how often users approve a generation without edits, measuring the quality of the "Ground Truth".
- **Preset K-Factor:** Identifies "Star Creators" by tracking how many unique users adopt a specific preset.

🛠️ The Technical Implementation Plan

To build this without increasing your AWS bill, we will use the **"Zero-Cost Pipeline"**:

Stage	Tool	Action
Extraction	**n8n (Tier 2)**	Runs on a cron job every 4 hours to query RDS and Stripe.
Warehouse	**Google Sheets**	Acts as a lightweight, free data warehouse for row-level metrics.
Visualization	**Looker Studio**	Connects to Sheets to provide the "COO Pulse" and "CTO Health"

12. The Stripe & Apple Pay Integration Layer

This formalizes how we move from "Free Trial" to "Paid User."

- **The "Credit Refill" Logic:** We will implement **Stripe Checkout** for one-time top-ups and **Stripe Billing** for monthly subscriptions.
- **The Workflow:**
    1. User clicks "Get Credits" in the Stitch Frontend.
    2. Amplify redirects to a Stripe-hosted checkout page (supports **Apple Pay**, Google Pay, and Credit Cards).
    3. On success, Stripe sends a **Webhook** to our n8n "Airlock".
    4. n8n validates the transaction and updates the `swift_credits` column in the **Supabase/RDS Profiles table**.

13. Security Mesh: Reference Protocol

We implement a **Zero-Trust Security Mesh** to harden the platform against "Vibe Coding" vulnerabilities (hallucinated code, slopsquatting).

- **Layer 1 (The Fortress):** AWS WAF (Web Application Firewall) on Amplify to block DoS and SQL injection.
- **Layer 2 (The Airlock):** All n8n webhooks require a `x-swiftlist-secret` header for validation.
- **Layer 3 (The Sanctuary): Row Level Security (RLS)** in the database ensures users can never query another's data.
- **Layer 4 (The Tunnel): Cloudflare Zero-Trust Tunnel** connects NUCs to AWS. No ports are opened on the home router.
- **Layer 5 (MCP Protocol):** AI agents interact with data via a restricted **Model Context Protocol (MCP)** tool-set with strict JSON schema validation.

The Technical Design Document has been updated to **Version 1.6** incorporating the "Creator Economy" and "Job Refinement" modules.

```
None
```

14. Version 1.6 Core Feature Modules

*The logic and specifications in this section supersede previous UI/UX sections regarding the Profile and Job Completion flows.***14.1 New Feature Module: The "Preset Marketplace" & Creator Economy**

- **Objective:** Transform SwiftList from a tool into a network-effect platform where users monetize their creativity.
- **Data Model Update (Users):**
  - Add `lifetime_credits_earned` (Integer).
  - Add `current_month_earnings` (Integer).
- **Data Model Update (Presets):**
  - Add `visibility` (Enum: private, public).
  - Add `usage_count` (Integer).
  - Add `creator_id` (Foreign Key -> Users).
- **UX Requirement (The "Save" Flow):**
  - **Trigger:** Occurs on the "Job Completed" screen *after* a successful generation.
  - **Modal UI:** "Save This Style as a Preset."
  - **Fields:** Name, Description, Categories (Tags), Visibility Toggle.
  - **Incentive Copy:** "Make Public to earn 1 Credit every time this preset is used."

14.2 New Feature Module: Advanced Job Management

- **The "Edit & Refine" Flow:**
  - **Problem:** Users currently have to start over and pay full price if they make a mistake.
  - **Solution:** Add an "Edit Settings" button to the Job Completed screen.
  - **Technical Logic:**
    - Retains the original `input_image_s3_url`.
    - Hydrates the "Job Setup" screen with the previous parameters.
    - **Incremental Billing:** Compare `new_cost` vs. `old_cost`. Only charge the difference (e.g., adding "Animated Spin" = +26 credits).
  - **Database:** Create a `parent_job_id` relationship to track iteration history.

14.3 UX/UI Polish & "Vibe" Requirements

- **Social Proof (Gallery):**
  - Display dynamic badges on preset cards: "🔥 Trending" (high velocity), "👑 Best Seller" (highest absolute volume), "⭐ Top Rated."
- **Profile Dashboard (The "Studio"):**
  - **Credit Visualization:** Replace standard text with a "Gas Gauge" or circular

progress ring for available credits.
    - **Earnings Card:** Dedicated section showing "Tokens Earned This Month" with percentage growth indicators to drive gamification.
- **Network Effects:**
    - **Referral Integration:** "Share & Earn" widget on the Job Completed screen (Double-sided reward: Referrer gets credits, Referee gets credits).

14.4 Updated Operational Constraints

- **Billing Logic:**
    - **Credit Top-Up:** Interstitial modal if user balance < job cost.
    - **Stripe Integration:** Use Stripe Elements for saved card management within the Profile settings.

15. The "Decider" Node: Dynamic Category Routing

- **Objective:** Automatically detect product categories to apply physics-correct rendering and marketplace compliance rules, specifically targeting failure modes found in generalist competitors (e.g., Pebblely).

15.1 Technical Implementation: The Classification Gate

- **Execution Node:** n8n "Model Selector" + Gemini 2.0 Flash.
- **Logic:** Every job begins with an instantaneous visual classification pass using Gemini 2.0 Flash to return a structured JSON object: `{ "category": "string", "risk_level": "high/low", "requires_precision_engine": boolean }`.

15.2 The Routing Matrix (n8n Switch Logic)

Based on the classification, the "Decider" node routes the job to one of the following specialized sub-workflows:

Detected Category	Specialized Engine Triggered	Technical Goal
Jewelry / Metals	**Jewelry Precision Engine**	Uses Gemini 2.5 Pro for 3D orientation & specular reflection mapping to prevent "AI melting"[1111].
Fashion / Apparel	**Runway/Sora Act-Two**	Solves "Uncanny Human" issues by using performance capture for natural fabric drape.

Glass / Liquid	**Refraction Engine**	Applies GPT Image 1.5 surgical masking to preserve transparency and ray-traced liquid physics[2222].
Furniture / Decor	**Spatial Grounding Engine**	Uses Gemini 2.5 Pro to detect the "Floor Plane" for perspective-correct shadows.
General Goods	**Standard Engine**	Routes to Stability AI/GPT Image 1.5 for maximum speed and 88%+ profit margins[3333].

15.3 High-Risk "Post-Pass" Logic

- **Trigger:** If `risk_level == "high"` (e.g., low-contrast inputs or dark backgrounds).
- **Action:** Trigger an additional **Mask Edge Refinement Node** using Stability AI's Ultra-High-Res API to clean up "rough edges" common in basic AI background removals.

15.4 Margin Protection (Routing Costs)

- **Precision Path:** (Jewelry/Glass/Fashion) utilizes Tier 3 logic (Gemini 2.5 Pro/Runway). These jobs are billed to the user at the **11-Credit Premium Rate** to maintain the 60% Net Margin target[4444].
- **Standard Path:** Utilizes Tier 1/2 logic (Gemini 2.0 Flash/GPT 1.5) to maximize platform profitability on simple tasks[5].

*** SPECIAL LOGIC ***

The Board has finalized the specialized technical logic for **Jewelry & eBay Compliance**. This is our "Category Killer" module designed to outperform generalist tools like Pebblely.

🏛️ The "eBay Standard" Compliance Engine

To ensure every jewelry asset is **Marketplace Ready**, the Board approves these mandatory

n8n logic gates.

1. The "Hero Shot" Rules (eBay Mandatory)

- **Requirement:** 1:1 Aspect Ratio.
- **Requirement:** Pure White (#ffffff) or Light Grey (#f0f0f0) background for the primary photo.
- **Logic:** SwiftList will automatically add **15% padding** around the jewelry to ensure it "fills the frame" without being cut off by the eBay UI.

2. Automated Variations

For every job, the system will output a standardized 4-pack:

1. **Marketplace Hero:** 1500px, White Background, 1:1.
2. **Detail Macro:** Macro-focus on the hallmark or gemstone facets.
3. **Lifestyle Setting:** The jewelry placed in the user's chosen "Vibe" (e.g., Teal Box or Urban).
4. **Social Asset:** High-contrast graphic with **legible text overlays** (e.g., "Authentic 18k Gold") powered by GPT Image 1.5's new text engine.

📊 Updated TDD Module: Jewelry & eBay Precision

Feature	Technical Implementation	Target Outcome
Geometry Guard	Gemini 2.5 Pro pre-analysis	Zero metal distortion/melting.
Specular Mapping	Environment-based reflection directives	Realistic metal shine.
Auto-Compliance	eBay standard sizing/padding logic	Instant listing approval.
Refraction Fix	High-energy ray-tracing prompt	Authentic diamond "fire."

	keywords	

Final Verdict: This module is the "Secret Sauce." By solving the **Jewelry Distortion** problem that Pebblely currently has, we capture the most profitable segment of the marketplace immediately.

Jewelry Rendering Guide - https://www.youtube.com/watch?v=UT4p-IFrbjI

This video deep-dives into why diamonds and reflective metals often fail in standard renders and validates our technical need for specialized geometry-aware logic.